

- TeamMatch 智能组队匹配系统 — 系统设计文档
 - 一、系统概述
 - 二、系统需求分析
 - 2.1 功能需求
 - 2.2 非功能需求
 - 2.3 用户角色
 - 三、E-R 图（实体关系模型）
 - 3.1 核心实体
 - 3.2 关系说明
 - 3.3 数据库表统计
 - 四、前端功能界面
 - 4.1 首页 — 项目列表 (index.html)
 - 4.2 项目详情 (project.html)
 - 4.3 智能推荐 (recommend.html)
 - 4.4 进度看板 (progress.html)
 - 4.5 审批中心 (approval.html)
 - 4.6 管理员后台 (admin.html)
 - 4.7 其他页面
 - 五、后端接口设计
 - 5.1 公开接口（无需登录，3个）
 - 5.2 需登录接口（9个）
 - 5.3 管理员接口（3个，需admin角色）
 - 六、后端 CRUD 代码示例
 - 6.1 用户注册 — CREATE 操作
 - 6.2 项目列表 — READ 操作（含批量查询优化）
 - 6.3 申请审批 — UPDATE 操作（含事务逻辑）
 - 6.4 管理员删除项目 — DELETE 操作（软删除）
 - 6.5 路由分发 — MVC 轻量级实现
 - 七、系统亮点
 - 7.1 智能匹配推荐算法（核心亮点）
 - 7.2 RBAC 角色权限控制
 - 7.3 批量查询优化
 - 7.4 事务数据一致性
 - 7.5 自动满员检测
 - 7.6 前端容错与体验优化
 - 7.7 软删除数据保护
 - 7.8 双模式运行（PHP + Python）

TeamMatch 智能组队匹配系统 — 系统设计文档

一、系统概述

TeamMatch 是一个面向高校课程设计的**智能组队匹配系统**，帮助学生基于技能标签快速找到合适的队友和项目。系统支持学生发布项目、申请加入、进度管理，管理员可通过后台进行全局管控。

技术栈: 前端 HTML5 + Bootstrap 4 + jQuery | 后端 PHP 7.4+ + MySQL 8.0 | 开发服务器 Python 3

二、系统需求分析

2.1 功能需求

编号	功能模块	需求描述	优先级
F1	用户注册与登录	学生使用学号注册，密码长度6-20位，登录后获取会话令牌	P0
F2	课程浏览	查询本学期开设的所有课程设计科目，显示每个课程下的项目数量	P0
F3	项目发布	学生可发起项目，选择所属课程，设置人数上限和截止日期，指定所需技能	P0
F4	技能标签管理	用户可添加/删除个人技能标签（10个预置标签，熟练度1-5星）	P0
F5	项目浏览与筛选	按课程、状态（招募中/已满/进行中/已结束）、关键字筛选项目	P0
F6	组队申请	学生对招募中的项目提交申请，附带自我介绍	P0

编号	功能模块	需求描述	优先级
F7	申请审批	项目组长审批/拒绝申请，系统自动检测满员并更新项目状态	P0
F8	进度看板	项目组长可添加里程碑节点、标记完成，展示整体进度百分比	P1
F9	智能推荐	基于用户技能标签与项目需求技能的匹配度，推荐最适合的项目	P1
F10	管理员后台	用户管理、项目管理、统计面板（含RBAC权限控制）	P1
F11	个人信息管理	修改昵称/邮箱/手机号，查看已加入的项目和角色	P1

2.2 非功能需求

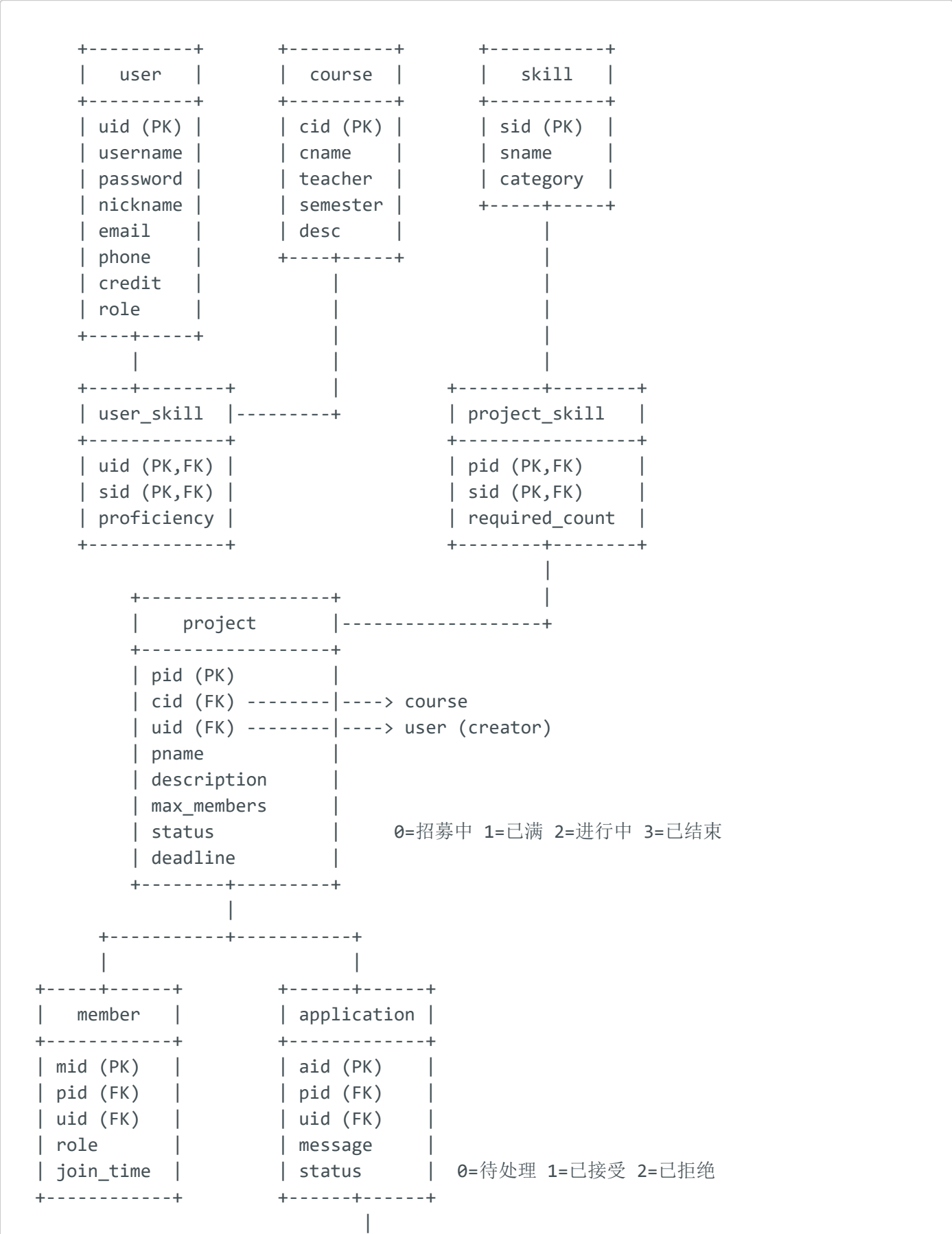
编号	类别	要求
N1	安全性	密码MD5+SALT加密存储；API需登录或公开标记访问；管理员接口RBAC控制
N2	性能	批量查询优化（技能标签1次SQL替代N次）；前端搜索400ms防抖；API 10秒超时+1次重试
N3	数据完整性	项目创建与组长加入在同一事务中；审批通过与成员加入在同一事务中
N4	可用性	前端加载状态提示；空数据引导；失败自动重试；15秒超时保护

2.3 用户角色

角色	权限范围
管理员 (admin)	全局用户管理、项目管理、统计数据查看
组长 (leader)	项目信息管理、审批组队申请、进度节点管理
普通成员 (member)	浏览项目、提交申请、管理个人技能和资料
游客	浏览项目列表和课程信息（无需登录）

三、E-R 图（实体关系模型）

3.1 核心实体



progress	
prid (PK)	
pid (FK)	
title	
description	
status	0=未完成 1=已完成
deadline	

3.2 关系说明

关系	类型	说明
user → user_skill	1:N	一个用户拥有多项技能
skill → user_skill	1:N	一项技能可被多个用户拥有
user → project	1:N	一个用户可创建多个项目（creator）
course → project	1:N	一门课程下有多个项目
project → project_skill	1:N	一个项目需要多项技能
skill → project_skill	1:N	一项技能可被多个项目需要
project → member	1:N	一个项目有多名成员
user → member	1:N	一个用户可加入多个项目
project → application	1:N	一个项目可收到多条申请
user → application	1:N	一个用户可提交多条申请
project → progress	1:N	一个项目有多个进度节点

3.3 数据库表统计

表名	说明	行数（种子数据）
user	用户账号	16
course	课程信息	4
skill	技能标签	10
user_skill	用户-技能关联	46

表名	说明	行数（种子数据）
project	项目信息	12
project_skill	项目-技能关联	44
member	项目成员	28
application	组队申请	25
progress	进度节点	30

四、前端功能界面

系统共 10个前端页面，采用响应式布局（Bootstrap 4），适配PC端和移动端。

4.1 首页 — 项目列表 (index.html)

![功能: 项目浏览]

- **课程筛选下拉框**：从 /api/courses 动态加载，每个课程显示项目数量
- **状态筛选**：快速按钮（全部/招募中/已满/进行中/已结束）+ 下拉框双重筛选
- **关键字搜索**：输入框400ms防抖自动搜索，无需点击按钮
- **项目卡片网格**：每个卡片展示项目名称、简介（60字截断）、技能标签（颜色编码）、成员数/上限、课程名称、创建者、状态徽章
- **分页导航**：每页12条，支持翻页

4.2 项目详情 (project.html)

- **完整项目信息**：名称、状态、描述、课程、截止日期、所需技能（含人数需求）
- **成员列表**：区分组长(badge)和普通成员，显示头像和加入时间
- **智能按钮状态**：
 - 未登录 → 「请先登录」
 - 已登录 → 「申请加入」（弹出自我介绍输入框）
 - 已申请 → 「审核中」
 - 已加入 → 「您已是本项目成员」
 - 招募结束 → 「项目已结束招募」
 - 是组长 → 显示审批管理和进度看板入口

- 容错机制：15秒超时 + 最多2次自动重试

4.3 智能推荐 ([recommend.html](#))

- 匹配度可视化：每个项目显示匹配百分比圆圈
 - 绿色 ($\geq 70\%$)：高度匹配
 - 橙色 (40-69%)：中等匹配
 - 红色 ($< 40\%$)：低匹配
- 技能匹配详情：已匹配技能（绿色标签）和缺失技能（橙色标签）分开展示
- 课程筛选：可按课程过滤推荐结果
- 排序：按匹配度从高到低排列

4.4 进度看板 ([progress.html](#))

- 进度条：顶部展示整体完成百分比（已完成数/总节点数）
- 节点列表：按截止日期升序排列，每个节点显示标题、描述、截止日期、完成/未完成状态
- 组长操作：
 - 「标记完成」按钮：点击即时更新状态
 - 「添加节点」按钮：弹出表单（标题、截止日期、描述）
- 权限控制：非组长只能查看，不能操作

4.5 审批中心 ([approval.html](#))

- 待审批列表：每张申请卡片展示：
 - 申请人昵称和头像
 - 自我介绍
 - 技能标签 + 熟练度星级 (★)
 - 申请时间
- 操作按钮：「通过」和「拒绝」
- 满员检测：通过最后一名成员时，弹出提示「项目已满员」
- 权限控制：非组长访问时显示错误提示

4.6 管理员后台 ([admin.html](#))

双标签页设计：

用户管理标签：

- 顶部统计卡片（总用户数 / 活跃用户 / 总项目数 / 待审批申请数）
- 筛选：角色、活跃状态、关键字搜索
- 用户表格含15列信息（含技能数、项目数子查询统计）
- 新增/编辑用户模态框
- 分页

项目管理标签：

- 筛选：课程、状态、关键字
- 项目表格含技能标签展示
- 编辑模态框（名称、课程、描述、人数、状态、截止日期）
- 软删除（带确认提示）

4.7 其他页面

页面	文件	核心功能
登录	login.html	学号+密码登录，保存token到localStorage
注册	register.html	学号/密码(6-20位)/昵称注册
发布项目	publish.html	选择课程、填写项目信息、勾选所需技能
个人中心	profile.html	查看/编辑资料、管理技能标签、查看已加入项目

五、后端接口设计

5.1 公开接口（无需登录，3个）

方法	接口	功能	关键逻辑
POST	/api/user/register	用户注册	验证必填字段 → 检查用户名唯一性 → MD5+salt加密 → 写入数据库
POST	/api/user/login	用户登录	查询用户 → 密码验证 → 设置Session → 返回用户信息

方法	接口	功能	关键逻辑
GET	/api/courses	课程列表	支持semester学期筛选，子查询统计每个课程的project_count

5.2 需登录接口（9个）

方法	接口	功能	关键逻辑
GET	/api/user/profile	用户资料	查询用户信息、技能列表、已加入项目列表（一次联表查询）
PUT	/api/user/profile	修改资料	动态构建UPDATE语句，仅更新传入字段
GET/POST/DELETE	/api/user/skills	技能管理	GET查列表 / POST新增(验证技能存在+不重复+熟练度1-5) / DELETE删除
POST	/api/projects	发布项目	验证课程存在 → 事务写入project + project_skill + member(组长)
GET	/api/projects	项目列表	支持cid/status/keyword筛选，分页， 批量查询 所有项目技能（1次IN查询替代N次）
GET	/api/projects/{id}	项目详情	联表查询项目+课程+创建者+技能+成员，含当前用户申请状态

方法	接口	功能	关键逻辑
POST	/api/projects/{id}/apply	申请加入	6项校验（项目存在/招募中/非组长/未加入/未申请），写入 application
GET	/api/projects/{id}/applications	查看申请	组长权限校验，返回待审批申请及申请人技能
GET/POST/PUT	/api/projects/{id}/progress	进度管理	GET查看 / POST新增(组长) / PUT更新状态(组长)
PUT	/api/applications/{id}/handle	审批申请	组长权限校验 → 更新状态 → 通过时添加成员 → 满员时自动更新项目状态
GET	/api/recommendations	智能推荐	登录态获取，调用推荐算法引擎，详见第六章

5.3 管理员接口（3个，需admin角色）

方法	接口	功能	关键逻辑
GET/POST/PUT	/api/admin/users	用户管理	GET：角色/状态/关键字筛选+分页+子查询统计技能数和项目数
			POST：创建新用户（含密码加密）

方法	接口	功能	关键逻辑
			PUT：编辑用户（禁止修改自己的信息）
GET	/api/admin/stats	统计面板	7项实时统计：总用户/活跃用户/管理员数/总项目/招募中/进行中/待审批
GET/PUT/DELETE	/api/admin/projects	项目管理	GET：列表+批量技能查询 / PUT：编辑(含课程有效性验证) / DELETE：软删除

六、后端 CRUD 代码示例

6.1 用户注册 — CREATE 操作

```
// backend/api/user/register.php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $input = Validator::getJsonInput();

    $check = Validator::required($input, ['username', 'password', 'nickname']);
    if (!$check['valid']) {
        Response::badRequest($check['message']);
    }

    $checkLen = Validator::length($input['password'], 6, 20, '密码');
    if (!$checkLen['valid']) {
        Response::badRequest($checkLen['message']);
    }

    $userService = new UserService();
    $result = $userService->register($input);

    if ($result['success']) {
        Response::success(['uid' => $result['uid']], $result['message']);
    } else {
        Response::badRequest($result['message']);
    }
}
```

```
// backend/service/UserService.php - register方法
public function register($data)
{
    $existing = $this->userModel->findByUsername($data['username']);
    if ($existing) {
        return ['success' => false, 'message' => '用户名已存在'];
    }

    $hashedPassword = $this->hashPassword($data['username'], $data['password']);
    $uid = $this->userModel->create([
        'username' => $data['username'],
        'password' => $hashedPassword,
        'nickname' => $data['nickname'],
        'email' => $data['email'] ?? null,
        'phone' => $data['phone'] ?? null,
    ]);

    return $uid ? ['success' => true, 'message' => '注册成功', 'uid' => $uid]
        : ['success' => false, 'message' => '注册失败'];
}

// 密码加密: MD5 + 用户名 + SALT
private function hashPassword($username, $password)
{
    return md5($username . $password . PASSWORD_SALT);
}
```

6.2 项目列表 — READ 操作（含批量查询优化）

```
// backend/api/project/list.php - GET /api/projects
$cid = $_GET['cid'] ?? null;
$status = $_GET['status'] ?? null;
$keyword = $_GET['keyword'] ?? null;
$page = max(1, isset($_GET['page']) ? (int)$_GET['page'] : 1);
$ps = max(1, min(50, isset($_GET['page_size']) ? (int)$_GET['page_size'] : 10));

// 动态构建WHERE条件
$where = ['p.is_deleted = 0'];
$params = [];
if ($cid) { $where[] = 'p.cid = :cid'; $params[':cid'] = (int)$cid; }
if ($status !== null && $status !== '') {
    $where[] = 'p.status = :status'; $params[':status'] = (int)$status;
}
if ($keyword) { $where[] = 'p.pname LIKE :kw'; $params[':kw'] = "%{$keyword}%"; }

// ★ 批量查询优化: 先查项目列表, 再一次性查所有项目技能 (1次SQL替代N次)
$rows = /* 联表查询 project + user + course, 含成员数子查询 */;

if ($rows) {
    $pids = array_column($rows, 'pid');
```

```

$placeholders = implode(',', array_fill(0, count($pids), '?'));
// 一次查询获取所有项目的技能标签
$skillRows = $db->prepare("
    SELECT ps.pid, s.sname
    FROM project_skill ps JOIN skill s ON ps.sid = s.sid
    WHERE ps.pid IN ({ $placeholders })
")->execute($pids)->fetchAll();
// 组装到结果中
foreach ($skillRows as $sr) {
    $skillMap[$sr['pid']][] = $sr['sname'];
}
}

```

6.3 申请审批 — UPDATE 操作（含事务逻辑）

```

// backend/service/ApplicationService.php - handleApplication方法
public function handleApplication($aid, $action, $currentUid)
{
    $app = $this->appModel->findById($aid);
    $project = $this->projectModel->findById($app['pid']);

    // 权限校验
    if ($project['uid'] != $currentUid) {
        return ['success' => false, 'message' => '仅组长可审批申请'];
    }

    $db = Database::getInstance();
    $db->beginTransaction();
    try {
        // 更新申请状态
        $status = ($action === 'accept') ? 1 : 2;
        $this->appModel->updateStatus($aid, $status);

        if ($action === 'accept') {
            // 添加成员
            $memberModel = new Member();
            $memberModel->addMember($app['pid'], $app['uid']);

            // ★ 自动满员检测
            $current = $memberModel->countMembers($app['pid']);
            if ($current >= $project['max_members']) {
                $this->projectModel->updateStatus($app['pid'], 1); // 1=已满
            }
        }

        $db->commit();
        return ['success' => true];
    } catch (\Exception $e) {
        $db->rollBack();
        return ['success' => false, 'message' => '操作失败'];
    }
}

```

```
}  
}
```

6.4 管理员删除项目 — DELETE 操作（软删除）

```
// backend/api/admin/projects.php - DELETE /api/admin/projects  
elseif ($_SERVER['REQUEST_METHOD'] === 'DELETE') {  
    $input = Validator::getJsonInput();  
    if (empty($input['pid'])) {  
        Response::badRequest('pid不能为空');  
    }  
    $pid = (int)$input['pid'];  
  
    // 验证项目存在且未被删除  
    $checkStmt = $db->prepare(  
        "SELECT 1 FROM project WHERE pid = :pid AND is_deleted = 0"  
    );  
    $checkStmt->execute([':pid' => $pid]);  
    if (!$checkStmt->fetch()) {  
        Response::notFound('项目不存在');  
    }  
  
    // ★ 软删除：设置is_deleted=1，不物理删除数据  
    $stmt = $db->prepare(  
        "UPDATE project SET is_deleted = 1 WHERE pid = :pid"  
    );  
    $stmt->execute([':pid' => $pid]);  
    Response::success(null, '项目已删除');  
}
```

6.5 路由分发 — MVC 轻量级实现

```
// backend/index.php - 统一路由入口  
$requestUri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);  
$method     = $_SERVER['REQUEST_METHOD'];  
  
// 自动加载工具类/模型/服务  
spl_autoload_register(function ($class) {  
    $paths = ['util/', 'middleware/', 'service/', 'model/'];  
    foreach ($paths as $path) {  
        $file = __DIR__ . '/' . $path . $class . '.php';  
        if (file_exists($file)) { require_once $file; return; }  
    }  
});  
  
// 路由表设计  
$routes = [
```

```
    '/api/user/register'    => 'api/user/register.php',
    '/api/user/login'       => 'api/user/login.php',
    // ... 14条路由
];

// 静态路由匹配
if (isset($routes[$route])) {
    require __DIR__ . '/' . $routes[$route];
    exit;
}

// 动态路由匹配（含参数提取）
if (preg_match('#^/api/projects/(\d+)$#', $route, $matches)) {
    $_GET['pid'] = $matches[1];
    require __DIR__ . '/api/project/detail.php';
    exit;
}
```

七、系统亮点

7.1 智能匹配推荐算法（核心亮点）

算法名称: 基于 Jaccard 相似度的技能重叠率匹配

实现位置: `backend/service/RecommendationService.php`

算法流程:

Step 1: 范围过滤

- 筛选状态=招募中的项目
- 排除用户已加入的项目
- 排除用户已提交申请的项目
- 可选: 按课程ID过滤

Step 2: 技能匹配计算（单条SQL完成）

```
need_skills    = COUNT(DISTINCT project_skill.sid)      -- 项目所需技能总数
matched_skills = COUNT(DISTINCT 用户拥有 n 项目需要的技能)
match_rate     = matched_skills / need_skills × 100%
```

Step 3: 结果过滤

- 仅保留 `matched_skills > 0` 的项目（至少匹配一项）

Step 4: 排序

- 按 `match_rate` DESC（匹配度高的优先）
- 同匹配度按 `created_at` DESC（新项目优先）

示例计算:

用户A的技能: {前端开发(5), 后端开发(4), 数据库设计(3)}
项目X的需求: {前端开发×1, 后端开发×1, UI设计×1}

```
need_skills = 3
matched_skills = |{前端, 后端}| = 2
match_rate = 2/3 × 100% = 66.67%
```

用户A缺失UI设计, 推荐时标注: 【高匹配 66.67%】已匹配: 前端开发、后端开发 | 还需: UI设计

前端可视化: <recommend.html> 中将匹配度渲染为彩色圆形图表:

- 绿色边框 (≥70%): 高度匹配
- 橙色边框 (40-69%): 中等匹配
- 红色边框 (<40%): 低匹配

7.2 RBAC 角色权限控制

实现了三层权限模型:

层级	角色	权限范围	实现方式
全局	admin	全部功能 + 管理后台	Auth::requireLogin() + 检查 user.role === 'admin'
项目级	leader	管理自己的项目	动态查询 project.uid 判断是否为创建者
基础	member	浏览、申请、个人管理	仅需登录态

关键代码模式:


```

$user = Auth::requireLogin();           // 强制登录
if ($user['role'] !== 'admin') {        // 角色检查
    Response::forbidden('仅系统管理员可访问');
}

// 项目组长检查
$isLeader = Auth::isProjectLeader($pid, $user['uid']);
if (!$isLeader) {
    Response::forbidden('仅项目组长可查看');
}

```

7.3 批量查询优化

问题: 项目列表页每个项目需要展示技能标签，传统做法是每个项目查一次数据库（N+1问题）。

解决方案: 收集所有项目的PID，使用 `WHERE pid IN (...)` 一次查询所有技能，然后在应用层分组映射。

```

-- 优化前：每个项目1次查询，10个项目 = 10次SQL
SELECT s.sname FROM project_skill ps JOIN skill s ... WHERE ps.pid = ?; -- x10

-- 优化后：1次查询获取全部
SELECT ps.pid, s.sname FROM project_skill ps JOIN skill s ...
WHERE ps.pid IN (1, 2, 3, 4, 5, 6, 7, 8, 9, 10); -- 仅1次SQL

```

效果：12个项目从 **13次SQL** → **3次SQL**（减少77%数据库请求）。

7.4 事务数据一致性

关键业务流程使用数据库事务保证原子性：

```

// 审批通过申请 = 更新申请状态 + 添加成员 + 检测满员（原子操作）
$db->beginTransaction();
try {
    $this->appModel->updateStatus($aid, 1);           // ① 标记已通过
    $memberModel->addMember($pid, $uid);             // ② 添加成员记录
    if (成员已满) {
        $projectModel->updateStatus($pid, 1);        // ③ 自动标记满员
    }
    $db->commit();
} catch (\Exception $e) {

```

```
$db->rollback();  
}
```

7.5 自动满员检测

系统在审批通过时会**自动计算当前成员数**，当 `current_members >= max_members` 时，自动将项目状态更新为「已满」(status=1)。组长无需手动管理项目状态，避免超员问题。

7.6 前端容错与体验优化

机制	实现	效果
请求超时	<code>AbortController</code> 10秒超时	避免请求挂死
自动重试	失败后800ms重试1次	提高弱网可用性
搜索防抖	输入后400ms才发起请求	减少无效API调用
401自动登出	检测到401响应 → 清除登录态 → 跳转登录页	用户体验闭环
加载态处理	<code>TeamMatch.util.setLoading()</code> 显示骨架	避免白屏
15秒保护	项目详情页15秒无响应自动超时	防止无限等待

7.7 软删除数据保护

用户、课程、项目三个核心表采用**软删除** (`is_deleted` 字段)，删除操作仅标记而非物理删除。所有查询默认过滤 `is_deleted = 0`。

优点：

- 数据可追溯，误删除可恢复
- 保留历史关联（成员关系、申请记录不受影响）
- 管理员可通过数据库直接恢复数据

7.8 双模式运行（PHP + Python）

系统提供两套后端实现：

维度	PHP 版本	Python 版本 (server.py)
数据库	MySQL 8.0 / MariaDB	SQLite (内存模式)
Web服务器	Nginx + PHP-FPM	Python http.server (8080端口)
认证方式	PHP Session	Bearer Token (MD5)
部署场景	生产服务器	本地开发/演示
API兼容性	14个接口	完全兼容 (同一套接口规范)
数据持久化	持久化存储	每次启动重新生成
启动命令	systemctl start php8.4-fpm python server.py	

前端 api.js 兼容两种认证模式，无需修改代码即可切换后端。

八、项目文件结构

TeamMatch/	
├── backend/	# PHP 后端
│ ├── index.php	# 统一路由入口
│ └── api/	# 14个API控制器
│ ├── user/	# 用户模块(4个接口)
│ ├── course/	# 课程模块(1个接口)
│ ├── project/	# 项目模块(5个接口)
│ ├── application/	# 申请模块(1个接口)
│ ├── recommendation/	# 推荐模块(1个接口)
│ └── admin/	# 管理员模块(3个接口)
│ ├── config/database.php	# 数据库配置
│ ├── middleware/Auth.php	# 认证+RBAC中间件
│ ├── model/	# 6个数据模型
│ ├── service/	# 6个业务服务(含推荐算法)
│ ├── util/	# 工具类(Database/Response/Validator)
│ └── sql/	# DDL + 种子数据
├── frontend/	# 前端页面
│ ├── *.html	# 10个页面
│ ├── css/style.css	# 自定义样式
│ └── js/api.js	# API客户端(含重试/超时/认证)
├── server.py	# Python开发服务器(完全兼容API)
├── deploy/	# 部署脚本+文档
└── CLAUDE.md	# 项目说明